Real-Time Circle Detection.

Improving Accuracy Using OpenCV.

Course: Computer Vision Tutorial.

Student: Almanahil Al Harthy 22-0501.

Date: 1st of April 2026.

1. Introduction

This report documents the development of a real-time circle detection system using Python and OpenCV. The objective was to build upon a basic detection approach and improve its accuracy.

2. Problems Faced

The base circle detection system had several weaknesses that were identified during testing:

- Unstable detection: the detected circle jumped between different shapes across frames, making it unreliable.
- False circles: many non-circular regions in the background were incorrectly detected as circles.
- Wrong position and radius: detected circles often did not match the actual circular object in the scene.
- Sensitivity to noise: small amounts of image noise caused incorrect edge detection, leading to wrong Hough Transform results.
- No visual feedback: the base code provided no information about detection quality, coordinates, or FPS.

3. Changes Made and Why

3.1 Improvement 1 — Better Preprocessing

A Gaussian blur is applied to smooth the image and suppress noise before circle detection.

3.2 Improvement 2 — Parameter Tuning

All six parameters of cv.HoughCircles() are exposed as interactive trackbars:

Parameter	Default Value	Effect
dp	1.2	Inverse resolution ratio of the accumulator. Lower = more precise.
minDist	150	Minimum distance between circle centres. Prevents duplicate detections.
param1	100	Upper threshold for the Canny edge detector used internally.
param2	50	Accumulator threshold. Higher = fewer but more confident circles.
minRadius	30	Ignores circles smaller than this radius.
maxRadius	300	Ignores circles larger than this radius.

Being able to tune these parameters live was critical for adapting to different objects and lighting conditions without stopping and restarting the program.

3.3 Improvement 3 — Rejecting False Circles

After detection, the code filters out any circle whose radius falls outside the valid min/max radius range. When multiple circles pass the filter, the code selects the most appropriate one using the logic: if there is no previous circle, the largest circle is selected. This approach reduces random false detections from background texture and edges.

3.4 Improvement 4 — Stability Across Frames

To prevent the selected circle from jumping between different objects across frames, the system keeps track of the previously detected circle. In each new frame, the circle closest in position to the previous one is selected. This creates a smooth tracking behaviour where the detection follows the same object continuously rather than switching randomly.

Additionally, an anti-flicker mechanism was added: if the circle is temporarily lost, the system holds the last known circle position for up to 8 frames before clearing it. This

prevents the "No circle detected" warning from flashing briefly when the detector misses a single frame

4. Enhancements Added

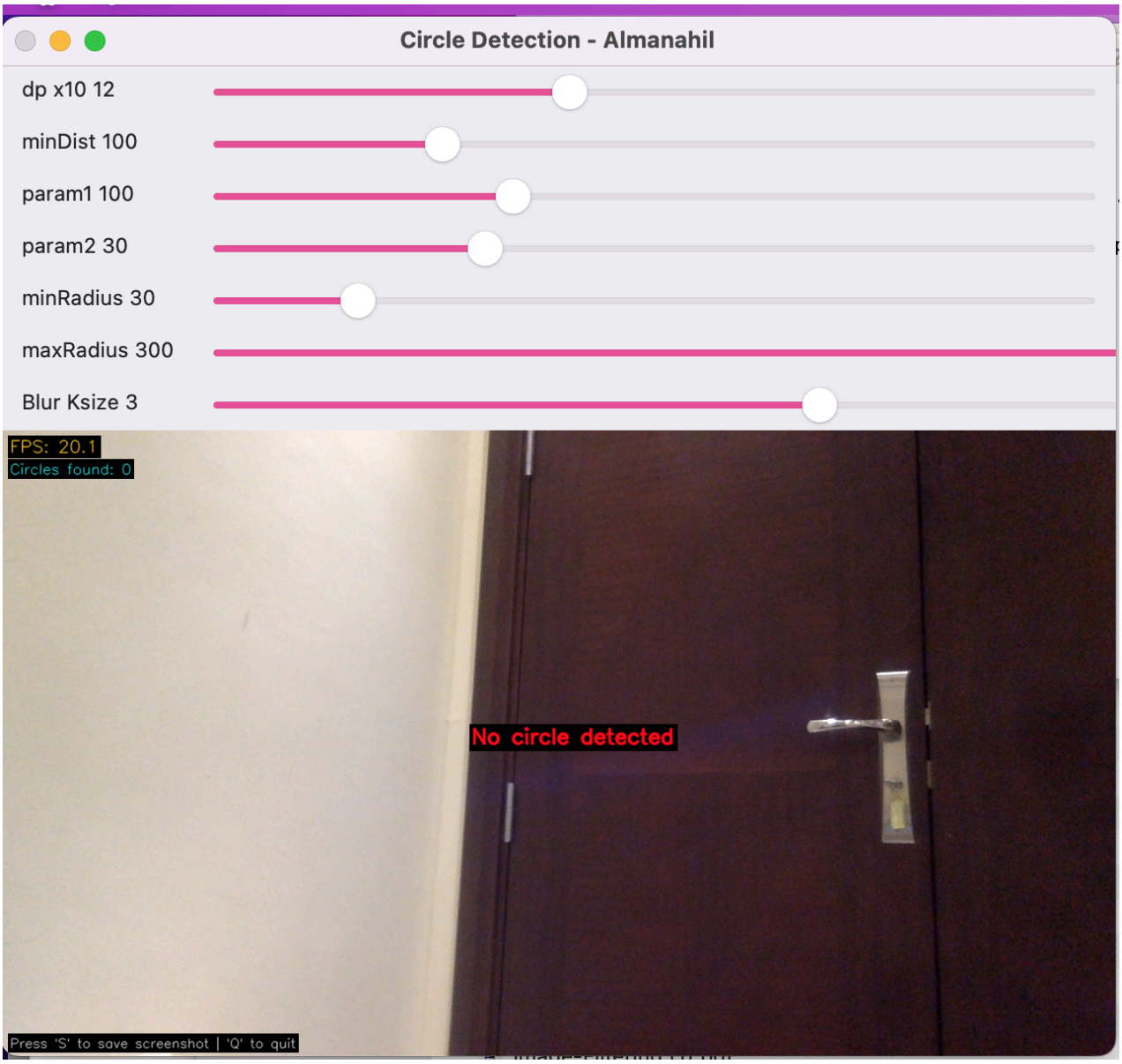
Enhancement	Description
Center coordinates (x, y)	Displayed on screen next to the detected circle in real time.
Radius display	Shows the radius in pixels next to the detected circle.
FPS counter	Displays the current frames per second in the top-left corner.
Circle count	Shows how many circles were detected by HoughCircles in the current frame.
No circle detected warning	A red warning message appears in the center of the screen when nothing is detected.
Screenshot saving	Pressing the 'S' key saves the current frame to a screenshots folder.
Interactive trackbars (Bonus)	All HoughCircles parameters and blur kernel are controllable via sliders in real time.

The most useful enhancement was the interactive trackbar sliders. Being able to adjust all parameters while the camera was running allowed immediate visual feedback and significantly reduced the time needed to tune the system for different objects and lighting conditions.

5. Results and Screenshots

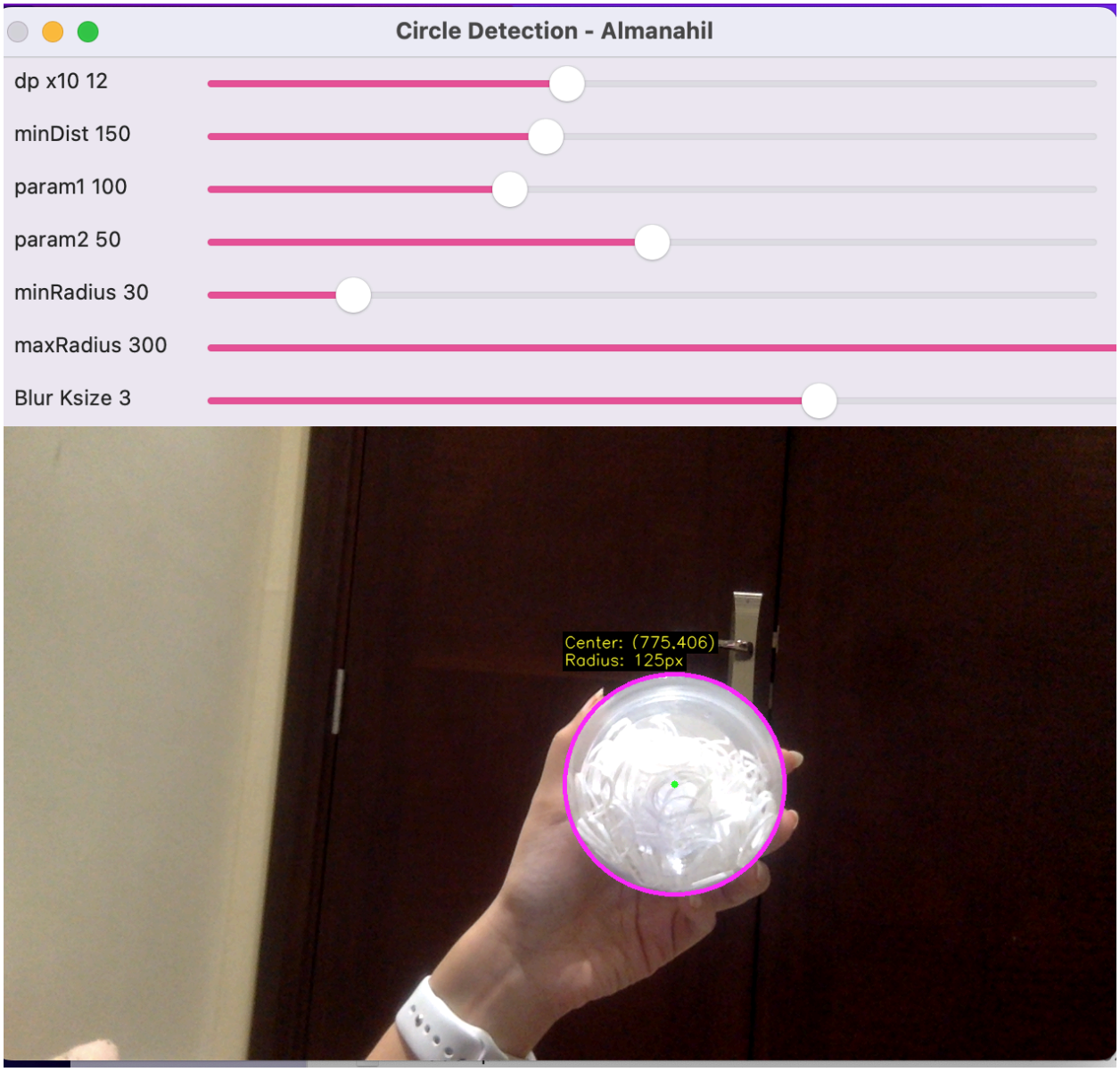
5.1 Before, There Was No Circle Detected

The first screenshot shows the system running with no circular object in the scene. The program correctly displays the "No circle detected" warning in red, with FPS running at 20.1 and 0 circles found. All trackbars are visible and the system is fully responsive.


Circle Detection - Almanahil
dp x10 12
minDist 100
param1 100
param2 30
minRadius 30
maxRadius 300
Blur Ksize 3
FPS: 20.1
Circles found: 0
No circle detected
Press 'S' to save screenshot | 'Q' to quit

5.2 After, Successful Circle Detection

Multiple circles were initially detected, but after tuning parameters (param2 and minDist), the system reliably selects a single correct circle.



6. Answers to Assignment Questions

Q1. Why is grayscale conversion used before circle detection?

HoughCircles works on single-channel images only. Converting to grayscale removes the three colour channels (R, G, B) and produces one intensity channel. This also reduces the amount of data to process, making detection faster and more focused on shape rather than colour.

Q2. Why is blur applied before using HoughCircles()?

Blur reduces high frequency noise in the image. Without blur, small intensity variations in the image are treated as edges by the internal Canny detector, producing many incorrect edge responses. Blurring smooths these out so that only real, significant edges remain, resulting in fewer false circle detections.

Q3. What is the effect of param2 on circle detection?

param2 is the accumulator threshold. Increasing it reduces false detections. In the final system, values around 50–60 provided stable and accurate detection.

Q4. Why do false circles appear in some frames?

False circles appear because the Hough Transform detects any pattern of edges that resembles a circular arc. Textured surfaces, printed labels, reflections, shadows, and background objects all produce edges that can accidentally match a circle. Increasing param2, restricting the radius range, and tracking the previous circle all help reduce this problem.

Q5. How did you improve the accuracy of your system?

Accuracy was improved through four main changes: applying Gaussian blur to reduce noise before detection, tuning all HoughCircles parameters interactively to match the target object, filtering out circles with unreasonable radii, and tracking the circle closest to the previous frame to prevent jumping. An anti-flicker mechanism also prevents brief false "no detection" frames.

Q6. Which enhancement gave the most useful result?

The interactive trackbar sliders were the most useful enhancement. They allowed all detection parameters to be adjusted in real time while watching the live camera feed. This made it possible to find the optimal

settings for each object quickly without stopping and restarting the program. Without this tool, parameter tuning would require multiple code edits and restarts.

Q7. What are the limitations of your final system?

The system has several limitations:

- The system does not work well on non-circular shapes or faces, as these produce too many irrelevant edges.
- Detection quality is sensitive to lighting conditions. Poor or uneven lighting can cause edges to disappear or multiply unexpectedly.
- The system can only reliably track one circle at a time.

7. Conclusion

Accuracy was improved by applying Gaussian blur, tuning HoughCircles parameters (especially param2 and minDist), filtering circles by radius, and tracking the previous circle for stability.